



JCS2201 PYTHON PROGRAMMING

UNIT – II CONTROL STRUCTURES AND STRINGS

UNIT 2 CONTROL STRUCTURES AND STRINGS SYLLABUS

Selective statements - if, if-else, nested if, if-elif ladder statements.

Iterative statements - while, for, Nested loops, else in loops, break, continue and pass statement

Strings: Formatting, Comparison, Slicing, Splitting, Stripping, Negative indices, String functions

CONTROL STRUCTURES

- Control flow refers to the sequence a program will follow during its execution.
- Conditions, loops, and calling functions influence how a Python program is controlled.
- 3 types of control structures in Python:
 1. Sequential - The default working of a program
 2. Selection - This structure is used for making decisions by checking conditions and branching
 3. Repetition - This structure is used for looping, i.e., repeatedly executing a certain piece of a code block.

3

CONTROL STRUCTURES

1. Sequential

Sequential statements are a set of statements whose execution process happens in a sequence.

This control structure represents the linear and sequential execution of code, where statements are executed one after another in the order they appear.

Eg.

```
a = 20
b = 10
c = a * b
print(f"The product of {a} and {b} = {c}")
```

OUTPUT

The product of 20 and 10 = 200

4

CONTROL STRUCTURES

2. Selection

- The statements used in selection control structures are also referred to as branching statements or decision control statements.
- They are used to make decisions.
- They alter the flow of the program based on the outcome of the decision.
- Different forms of selection statements in Python:
 - ✓ Simple if
 - ✓ if-else
 - ✓ The nested if
 - ✓ The complete if-elif-else ladder

5

CONTROL STRUCTURES

3. Repetition

- Repetition structure can be used to repeat a certain set of statements.
- It is also called as looping structure.
- Different repetition statements in Python:
 - ✓ The for loop
 - ✓ The while loop

6

SELECTION STRUCTURES – *if*

Simple 'if'

- It has only one condition to test in a basic if statement.
- If the condition is true, the indented statement runs. If not, nothing happens

SYNTAX:

```
if condition:
    statement-1
    statement-2
    :
    :
    statement-n
```



FLOWCHART

Example:

```
x=int(input("Enter x: "))
if x > 0:
    print(x, 'is positive')
    y = x * x
    print(y)
print('Bye')
```

OUTPUT 2

```
Enter x: -15
Bye
```

OUTPUT 1

```
Enter x: 10
10 is positive
100
Bye
```

SELECTION STRUCTURES – *if*

- **Eg. program** – Python program to generate a bill for 3 items purchased. Provide a 10% discount if the bill amount exceeds Rs. 2000.

EXAMPLE PROGRAM

```
a,b,c=map(int,input('Enter price for 3 items: ').split())
bill = a+b+c
if bill > 2000:
    print("You have a discount..\N{slightly smiling face}")
    discount = bill * 10/100
    bill -= discount
print("Here is your bill. Please pay Rs.", bill)
print("THANK YOU VISIT AGAIN")
```

SELECTION STRUCTURES – ‘if’

OUTPUT 1

Enter price for 3 items: 20 30 40
Here is your bill. Please pay Rs. 90
THANK YOU VISIT AGAIN

OUTPUT 2

Enter price for 3 items: 800 600 900
You have a discount.. 😊
Here is your bill. Please pay Rs. 2070.0
THANK YOU VISIT AGAIN

9

SELECTION STRUCTURES – ‘if-else’

‘if - else’

- When there are two possibilities if-else statement is used.
- The condition determines which possibility is executed.
- If the condition is true, the true block is executed.
- If the condition is false, the false block is executed.
- In both the cases, the statement next to if block is executed.
- The true block and the false block can contain one or more than one statement.

10

SELECTION STRUCTURES – ‘if-else’

FLOWCHART

SYNTAX:

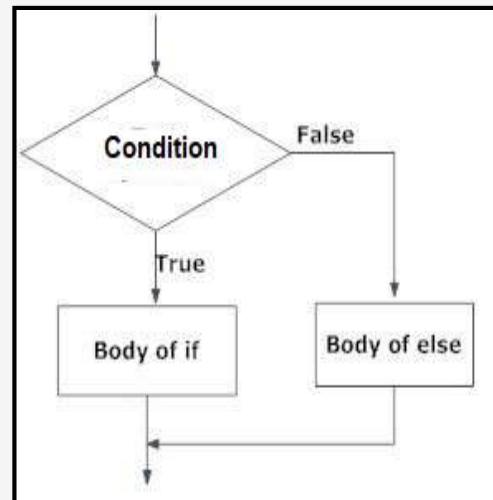
if condition:

 True block

else:

 False block

statement-outside if



11

SELECTION STRUCTURES – ‘if-else’

EXAMPLE PROGRAM

Example:

```

a,b=map(int, input('Enter 2 values: ').split())
if a>b:
    print(f"{a} is bigger")
else:
    print(f"{b} is bigger")
print("Thank you")
  
```

Python program to
find the bigger of 2
numbers

OUTPUT

```

Enter 2 values: 20 45
45 is the bigger
Thank you
  
```

12

SELECTION STRUCTURES – ‘if-else’

EXAMPLE PROGRAM

Example:

```
mark = int(input('Enter your mark: '))
if mark >= 50:
    print("PASSED")
else:
    print("FAILED")
```

Python program to check a student result as passed or failed

OUTPUT

Enter your mark: 70
PASSED

13

SELECTION STRUCTURES – ‘if-else’

‘if - else’

- Eg. program – Python program to handle a withdrawal transaction. Let the balance amount available be Rs. 10000 and the minimum balance to be maintained be Rs. 500.

```
balance = 10000
amount = int(input('How much amount do you want to withdraw? : '))
if balance - amount >= 500:
    balance -= amount
    print(f"Rs. {amount} debited. Your balance amount = Rs.{balance}")
else:
    print("Your balance is low. Withdrawal not possible")
    print("Your balance amount = Rs.{balance}")
print("THANK YOU.. VISIT AGAIN!")
```

SELECTION STRUCTURES – ‘if-else’

OUTPUT 1

How much amount do you want to withdraw? : 4000
Rs. 4000 debited. Your balance amount = Rs.6000
THANK YOU.. VISIT AGAIN!

OUTPUT 2

How much amount do you want to withdraw? : 9750
Your balance is low. Withdrawal not possible
Your balance amount = Rs.10000
THANK YOU.. VISIT AGAIN!

15

SELECTION STRUCTURES – ‘nested if’

‘nested if’

- Using an ‘if’ or ‘if-else’ statement inside another ‘if’ or ‘if-else’ statement is known as nested if.
- The inner if condition will be checked only if outer if condition is true and so multiple conditions have to be satisfied.

16

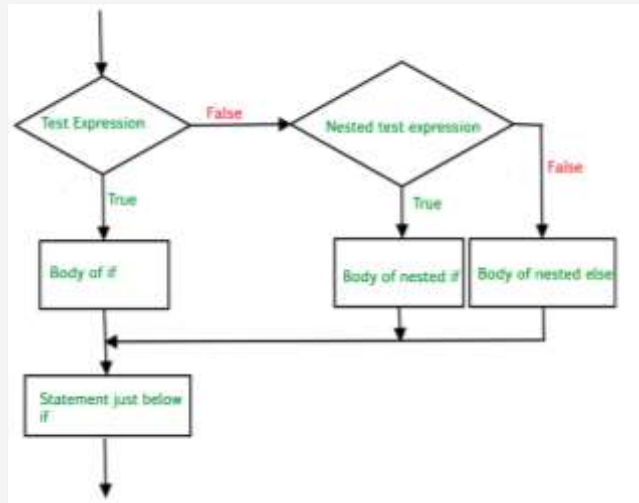
SELECTION STRUCTURES – ‘nested if’

FLOWCHART

SYNTAX:

```

if condition1:
    Statement block1
else:
    if condition2:
        Statement block2
    else:
        Statement block3
statement-outside if
  
```



17

SELECTION STRUCTURES – ‘nested if’

EXAMPLE PROGRAM -

```

a,b,c=map(int, input('Enter 3 values: ').split())
if a>b:
    if a>c:
        print(f"{a} is the biggest")
    else:
        print(f"{c} is the biggest")
else:
    if b>c:
        print(f"{b} is the biggest")
    else:
        print(f"{c} is the biggest")
  
```

Python program to find the biggest of 3 numbers

OUTPUT

Enter 3 values: 20 45 33
45 is the biggest

18

SELECTION STRUCTURES – ‘nested if’

‘nested if’

- Eg. program – Python program to check if a number is positive, negative or zero.

```
num = int(input('Enter a number: '))
if num >= 0:
    if num == 0:
        print(f"{num} is ZERO")
    else:
        print(f"{num} is POSITIVE")
else:
    print(f"{num} is NEGATIVE")
```

OUTPUT 1

Enter a number: 20
20 is POSITIVE

OUTPUT 2

Enter a number: 0
0 is ZERO

OUTPUT 3

Enter a number: -56
-56 is NEGATIVE

19

SELECTION STRUCTURES – ‘nested if’

Eg. program – Python program to declare result for a student. If mark is 50 or more than 50, declare the result as ‘passed’; otherwise as ‘failed’. Maximum mark is 100. Negative mark is not valid.

```
mark = int(input('Enter your mark: '))
if mark >= 0 and mark <= 100:
    if mark >= 50:
        print("PASSED")
    else:
        print("FAILED")
else:
    print("Invalid mark")
```

OUTPUT 1

Enter your mark: 28
FAILED

OUTPUT 2

Enter your mark: 102
Invalid mark

OUTPUT 3

Enter your mark: 73
PASSED

20

SELECTION STRUCTURES – ‘nested if’

Eg. program – Python program for a validated user registration with a user name, valid email id and a password.

```
name = input("Enter your name: ")
email = input("Enter your email address: ")
password = input("Enter your password: ")
if name == "":
    print("Name cannot be empty.")
else:
    if "@gmail.com" not in email:
        print("Use gmail id only.")
    else:
        if len(password) < 8:
            print("Password must be at least 8 characters long.")
        else:
            print("Registration successful.")
```

SELECTION STRUCTURES – ‘nested if’

Eg. program – Python program to register a user with name, valid email id and a password.

OUTPUT 1

```
Enter your name: qwerty
Enter your email address: asdf@yahoo.com
Enter your password: 123456
Use gmail id only.
```

OUTPUT 2

```
Enter your name: Leya
Enter your email address: asdf@gmail.com
Enter your password: abcd1234
Registration successful.
```

SELECTION STRUCTURES – ‘if – elif - else’

- It enables us to check multiple conditions and execute the specific block of statements depending upon the true condition among them.
- If none of the conditions is satisfied the default block that follows ‘else’ statement is executed – but ‘else’ is optional.
- Any number of elif statements can be used in a program depending upon the need.
- The elif statement works like an if-else-if ladder statement in C.

23

SELECTION STRUCTURES – ‘if – elif - else’

SYNTAX:

if condition1:

 Statement block1

elif condition2:

 Statement block2

Elif condition3:

 Statement block3

·

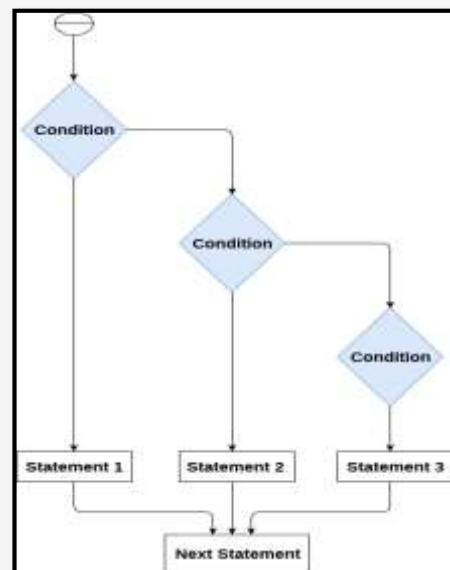
·

else:

 Statement block-n

statement-outside if

FLOWCHART



24

SELECTION STRUCTURES – ‘if – elif - else’

Eg. program –
Python program
to display grade
based on marks
scored.

OUTPUT

Enter the marks: 66
You scored grade B.

```
marks = int(input("Enter the marks: "))
if marks > 85 and marks <= 100:
    print("Congrats ! you scored grade A.")
elif marks > 60 and marks <= 85:
    print("You scored grade B.")
elif marks > 40 and marks <= 60:
    print("You scored grade C.")
elif (marks > 30 and marks <= 40):
    print("You scored grade D.")
else:
    print("Sorry you have failed")
```

25

SELECTION STRUCTURES – ‘if – elif - else’

Eg. program –
Python program
to read user's age
and print the
corresponding
age group.

OUTPUT

Enter your age: 24
You are an adult.

```
age = int(input("Enter your age: "))
if age > 60:
    print("A senior citizen.")
elif age >= 20:
    print("An adult.")
elif age >= 13:
    print("A teenager.")
elif age >= 1:
    print("A child.")
else:
    print("A baby.")
```

26

SELECTION STRUCTURES – 'if – elif - else'

Python program to design a simple calculator

```
print("SIMPLE CALCULATOR")
num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))
print("1 Addition")
print("2 Subtraction")
print("3 Multiplication")
print("4 Division")
print("5 Floor division")
print("6 Modulo division")
print("7 Exponent")
operation = input("Enter your choice of operation: ")
if operation == '1':
    print(f"{num1} + {num2} = {num1 + num2}")
elif operation == '2':
    print(f"{num1} - {num2} = {num1 - num2}")
elif operation == '3':
    print(f"{num1} * {num2} = {num1 * num2}")
```

```
elif operation == '4':
    if num2 == 0:
        print("Error: Cannot divide by zero.")
    else:
        result = num1 / num2
elif operation == '5':
    if num2 == 0:
        print("Error: Cannot divide by zero.")
    else:
        print(f"{num1} // {num2} = {num1 // num2}")
elif operation == '6':
    if num2 == 0:
        print("Error: Cannot divide by zero.")
    else:
        print(f"{num1} % {num2} = {num1 % num2}")
elif operation == '7':
    print(f"{num1} ** {num2} = {num1 ** num2}")
else:
    print("Invalid operation.")
```

27

SELECTION STRUCTURES

OUTPUT:

```
SIMPLE CALCULATOR
Enter first number: 22
Enter second number: 3
1 Addition
2 Subtraction
3 Multiplication
4 Division
5 Floor division
6 Modulo division
7 Exponent
Enter your choice of operation: 1
22 + 3 = 25
```

Programs for practice

<https://www.pyforschool.com/assignment/conditional.html>

28

ITERATIVE STATEMENTS – ‘for’ LOOP

- A for loop is used for iterating over a sequence (that is either a range of sequential values, a list, a tuple, a dictionary, a set, or a string).
- With the for loop we can execute a set of statements, for a certain number of times or once for each item in a list, tuple, set etc.

SYNTAX:

```
for iterator_variable in sequence:
    statement block
```

29

ITERATIVE STATEMENTS – ‘for’ LOOP

LOOPING USING RANGE FUNCTION

Default start value is 0.
Iterator_variable takes
values from 0 to stop-1

```
for iterator_variable in range(stop):
    statement block
```

```
for iterator_variable in range(start, stop):
    statement block
```

Iterator_variable takes
values from start to
stop-1

Iterator_variable takes
values from start to
stop-1, incremented by
step value

```
for iterator_variable in range(start, stop, step):
    statement block
```

ITERATIVE STATEMENTS – ‘for’ LOOP

LOOPING USING RANGE FUNCTION

```
for i in range(5):
    print(i)
```

OUTPUT

```
0
1
2
3
4
```

```
for i in range(3, 10):
    print(i)
```

OUTPUT

```
3
4
5
6
7
8
9
```

```
for i in range(3, 16, 4):
    print(i)
```

OUTPUT

```
3
7
11
15
```

31

ITERATIVE STATEMENTS – ‘for’ LOOP

FLOWCHART FOR A LOOP THROUGH A SEQUENCE



LOOPING THROUGH A STRING

```
for c in "fruits":
    print(c)
```

OUTPUT

```
f
r
u
i
t
s
```

LOOPING THROUGH A LIST

```
colors = ["red", "orange", "blue"]
for c in colors:
    print(c)
```

OUTPUT

```
red
orange
green
```

32

ITERATIVE STATEMENTS – 'for' LOOP

Write the following Python programs:

1. Python program to display the first n natural numbers.
2. Python program to display the even numbers from 1 to n.
3. Python program to find the sum of first n natural numbers.
4. Python program to find the sum of n natural numbers.
5. Python program to find the factorial of a number.
6. Python program to find the exponent of a number - a^n .
7. Python program to display the multiplication table of a number.
8. Python program to display the Fibonacci series.

33

ITERATIVE STATEMENTS – 'for' LOOP

Python program to display the first n natural numbers.

```
n = int(input("Enter the limit: "))
for i in range(1, n+1):
    print(i)
```

OUTPUT

Enter the limit: 4

1
2
3
4

Python program to find the sum of first n natural number

```
sum = 0
n = int(input("Enter the limit: "))
for i in range(n):
    sum = sum + (i+1)
print("SUM = ", sum)
```

OUTPUT

Enter the limit: 12
SUM = 78

34

ITERATIVE STATEMENTS – ‘for’ LOOP

Python program to find the factorial of a number

```
fact = 1
n = int(input("Enter a number: "))
for i in range(1, n+1):
    fact *= i
print(f"Factorial of {n} = {fact}")
```

OUTPUT

Enter a number: 5
Factorial of 5 = 120

Python program to display the even numbers from 1 to n

```
n = int(input("Enter the limit: "))
for i in range(1, n+1):
    if i%2 == 0:
        print(i)
```

OUTPUT

Enter the limit: 9
2
4
6
8

35

ITERATIVE STATEMENTS – ‘for’ LOOP

Python program to print Fibonacci series

Fibonacci series: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

If n=7, the output is 0, 1, 1, 2, 3, 5, 8

```
a, b = 0, 1
n = int(input("Enter the number of values: "))
print(a)
print(b)
for i in range(n-2):
    c = a + b
    a = b
    b = c
    print(c)
```

OUTPUT

Enter the number of values: 7
0
1
1
2
3
5
8

ITERATIVE STATEMENTS – ‘while’ LOOP

- A while loop is used to execute a block of statements repeatedly as long as a condition remains true.
- When the condition becomes false, the line immediately after the loop in the program is executed

SYNTAX:

```
while condition:
    statement block
Statement outside while
```

37

ITERATIVE STATEMENTS – ‘while’ LOOP

Example – Python program to display Hello and count 5 times

```
count = 1
while count <= 5:
    print("Hello ", count)
    count = count + 1
```

OUTPUT

```
Hello 1
Hello 2
Hello 3
Hello 4
Hello 5
```

Example – Python program to find the sum of digits of a number

```
sum = 0
n = int(input("Enter a number: "))
while n > 0:
    rem = n % 10
    sum += rem
    n //= 10
print("Sum of digits = ", sum)
```

OUTPUT

```
Enter a number: 3815
Sum of digits = 17
```

38

ITERATIVE STATEMENTS – ‘while’ LOOP

Example – Python program to find the reverse of a number

```
sum = 0
n = int(input("Enter a number: "))
n1 = n
while n > 0:
    rem = n % 10
    sum = sum * 10 + rem
    n //= 10
print(f"Reverse of {n1} = {sum}")
```

OUTPUT

Enter a number: 3815
Reverse of 3815 = 5183

39

ITERATIVE STATEMENTS – ‘while’ LOOP

Example – Python program to check if a number is a Palindrome or not

```
sum = 0
n = int(input("Enter a number: "))
n1 = n
while n > 0:
    rem = n % 10
    sum = sum * 10 + rem
    n //= 10
if sum == n1:
    print(f"{n1} is a Palindrome")
else:
    print(f"{n1} is not a Palindrome")
```

OUTPUT

Enter a number: 38183
38183 is a Palindrome

40

NESTED LOOPS

- Nested 'for' loop

SYNTAX:

initialization

```
for iterator_variable1 in sequence1: → Outer loop
    statement_block1
    for iterator_variable2 in sequence2: → Inner loop
        statement_block2
```

Statement_block1 and all the iterations of inner for loop are executed for each iteration of the outer 'for' loop

NESTED LOOPS

- Nested 'for' loop

```
for i in range(5):
    for j in range(3):
        print(j, end = ' ')
    print()
```

OUTPUT

```
0 1 2
0 1 2
0 1 2
0 1 2
0 1 2
```

In this example,
When i is 0, j = 0, 1, 2
When i is 1, j = 0, 1, 2
This is repeated till i is 4

NESTED LOOPS

Python program to display the given output

```
n = int(input("Enter a number: "))
for i in range(1, n+1):
    for j in range(i):
        print(' * ', end=' ')
    print()
```

OUTPUT

```
*
*  *
*  *  *
*  *  *  *
*  *  *  *  *
*  *  *  *  *  *
```

45

NESTED LOOPS

- Nested 'while' loop

SYNTAX:

```
initialization
while(condition):
    initialization of inner loop
    while(condition):
        Update expression of inner loop
    Update expression of outer loop
```

→ Outer loop

→ Inner loop

46

NESTED LOOPS

- Nested 'while' loop

```
i = 1
while i<=6:
    j = 1
    while j<=5:
        print(i*j, end='  ')
        j += 1
    print()
    i += 2
```

OUTPUT

```
1 2 3 4 5
3 6 9 12 15
5 10 15 20 25
```

In this example,
When i is 1, j = 0 to 5
When i is 3, j = 0 to 5
When i is 5, j = 0 to 5

47

UNCONDITIONAL STATEMENTS

- Looping statements will make a part of the program to execute repeatedly – either for a certain number of times / until a condition remains true
- If we have to quit/exit the loop in the middle (abrupt termination of loop) or skip to the next iteration we can use unconditional statements.
- Unconditional statements allow to direct the program's flow to another part of a program without evaluating conditions.

48

UNCONDITIONAL STATEMENTS

- The Unconditional statements available in Python are

(1) **break**



Used to terminate loops

(2) **continue**



Used to terminate an iteration

(3) **pass**



Used to do nothing

(4) **return**



Used in functions

49

'break' STATEMENT

- 'break' allows to exit the current loop when an external condition is met.
- When 'break' is executed, the remaining statements inside the loop are ignored.
- 'break' statement can be used with both for loops and while loops.
- In a nested loop, 'break' will stop execution of the loop inside which 'break' is used.
- 'break' causes a forward jump – the next statement to be executed is after the loop.

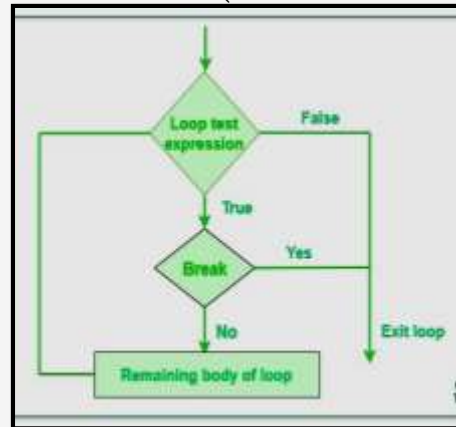
50

'break' STATEMENT

- Statements 1 are executed.
- If condition becomes true, 'break' is executed.
- When 'break' is executed, the statements 2 are ignored.
- Control jumps to execute statements 3 (outside the loop)

SYNTAX:

```
for / while loop:
    statements 1
    if condition:
        break
    statements 2
statements 3
```



51

'break' STATEMENT

Python program to print natural numbers and stop if it reaches 5

```
n = int(input("Enter the limit: "))
for i in range(1, n+1):
    if i == 5:
        break
    print(i)
```

OUTPUT

```
Enter the limit: 12
1
2
3
4
```

'continue' STATEMENT

- 'continue' allows to end the current iteration in a loop, when an external condition is met.
- When 'continue' is executed, the remaining statements inside the loop are ignored .
- 'continue' statement can be used with both for loops and while loops.
- In a nested loop, 'continue' will stop execution of the loop inside which it is used.
- 'continue' causes a backward jump – the next statement to be executed is the first statement in the loop.

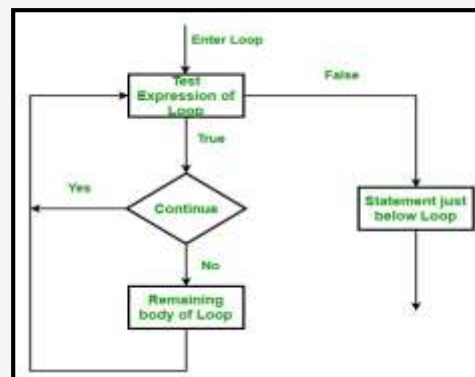
53

'continue' STATEMENT

- Statements 1 are executed.
- If condition becomes true, 'continue' is executed.
- When 'continue' is executed, the current iteration is stopped and statements 2 are ignored.
- Control jumps to execute statements 1

SYNTAX:

```
for / while loop:
    statements 1
    if condition:
        continue
    statements 2
statements 3
```



54

'continue' STATEMENT

Python program to print natural numbers in a range that are not divisible by 5

```
n1 = int(input("Enter the lower limit: "))
n2 = int(input("Enter the upper limit: "))
for i in range(n1, n2):
    if i % 5 == 0:
        continue
    print(i)
```

OUTPUT

```
Enter the lower limit: 3
Enter the upper limit: 12
3
4
6
7
8
9
11
```

'pass' IN PYTHON

- 'pass' is a null statement.
- It is like a comment, but it is not ignored by the interpreter.
- When the user does not know what code to write, user simply places a 'pass' at that line.
- It can be placed where empty code is not allowed, like in loops, function definitions, class definitions, or in if statements.
- It can be used as a placeholder for future code.

'pass' IN PYTHON

Example program without using 'pass'

```
n=10
if n == 10:
    # don't know what to write
    print("Hello World")
```

OUTPUT

IndentationError: expected an indented block after 'if' statement on line 1

Example program using 'pass' and avoiding error

```
n = 10
if n == 10:
    pass
print("Hello World")
```

OUTPUT

Hello World

'else' IN LOOPS

- 'else' with loop is used with both while and for loop.
- The 'else' block is executed at the end of loop → when the given loop condition is false then the else block is executed.
- The else clause is executed only if the loop completes normally
- It is not executed if the loop was terminated by a break (or) it was never entered.
- It is used to express post-loop actions

SYNTAX:

```
for loop:
    statements 1
else:
    statements 2
```

SYNTAX:

```
while condition:
    statements 1
else:
    statements 2
```

'else' IN LOOPS**Example programs using 'else' in for loop**

```
n = 4
for i in range(1, n):
    print(i)
else:
    print("PYTHON")
```

OUTPUT

```
1
2
3
PYTHON
```

Since the loop is completely executed here,
the else block is also executed finally.

59

'else' IN LOOPS**Example programs using 'else' in for loop**

```
n = 4
for i in range(1, n):
    if i > 4:
        break
    print(i)
else:
    print("HELLO")
```

Since break is not
executed here, the else
block is executed

OUTPUT

```
1
2
3
HELLO
```

```
n = 24
for i in range(1, n):
    if i > 4:
        break
    print(i)
else:
    print("HELLO")
```

Since break is
executed here,
the else block is
not executed

OUTPUT

```
1
2
3
4
```

'else' IN LOOPS**Example program using 'else' in while loop**

```
i = 1
while i <= 3:
    print(i)
    i = i + 1
else:
    print("HELLO")
```

OUTPUT

```
1
2
3
HELLO
```

Since the loop is completely executed here, the else block is also executed finally.

61

'else' IN LOOPS**Python program to check if a number is prime or not**

```
n = int(input("Enter a number: "))
for i in range(2, n):
    if n % i == 0:
        print(f"{n} is not a prime number")
        break
else:
    print(f"{n} is a prime number")
```

OUTPUT

```
Enter a number: 37
37 is a prime number
```

```
Enter a number: 27
27 is not a prime number
```

STRINGS

- A String is a data structure in Python that represents a sequence of characters.
- It is an immutable data type, meaning that once you have created a string, you cannot change it.
- Strings are used widely in many different applications, such as storing and manipulating text data, representing names, addresses, and other types of data that can be represented as text.
- Strings must be enclosed within single quotes ' ' (or) double quotes " " (or) triple quotes """
- Eg. name = "Kannan"

63

STRINGS

Few more examples:

```
>>> name = "Kannan"
```

```
>>> gr = 'A'
```

```
>>> dept = 'ECE'
```

```
>>> employee_id = "IOB157"
```

```
>>> print("My name is ", name)
```

My name is Kannan

```
>>> print(name + " scored" + gr + " grade")
```

Kannan scored A grade

```
Eg. >>> print("Hello")
```

Hello

```
>>> print("Hello" + "world")
```

Helloworld

'+' operator is used to concatenate strings

64

STRINGS

Accessing strings

Individual characters of a String can be accessed by using the method of Indexing.

Characters	P	Y	T	H	O	N		P	R	O	G	R	A	M
Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13

```
>>> str1 = 'PYTHON PROGRAM'
```

```
>>> print(str1)
```

```
PYTHON PROGRAM
```

```
>>> print(str1[2])
```

```
T
```

```
>>> str1[3] = 'G'
```

```
TypeError: 'str' object does not support item assignment
```

65

FORMATTING IN STRINGS

String formatting allows to create dynamic strings by combining variables and values.

Different ways to perform string formatting in Python

- Formatting with % operator.
- Formatting with format() string method.
- Formatting with string literals, called f-strings.
- Formatting with center() string method.

66

FORMATTING IN STRINGS

Formatting with % operator

The modulo %s is also known as the "string-formatting operator". It acts like a placeholder for a string or string variable

```
print("I %s for ice cream" %'scream')
```

```
print("I %s for ice %s"%( 'scream', 'cream'))
```

```
a = 'scream'
```

```
print("I %s you %s for ice %s"%(a, a, 'cream'))
```

OUTPUT

I scream for ice cream

I scream for ice cream

I scream you scream for ice cream

FORMATTING IN STRINGS

Formatting with format() string method

One or more replacement fields and placeholders defined by a pair of curly braces {} into a string and calling the str.format().

```
print('Apple is {} in color' . format('red'))
```

```
print('{2} is {0} in {1}' . format('red', 'color', 'Apple'))
```

```
print("{a} is {a} in {b}" . format(a='orange', b='color'))
```

OUTPUT

Apple is red in color

Apple is red in color

orange is orange in color

FORMATTING IN STRINGS

Formatting with string literals, called f-strings

F-strings provide a concise and convenient way to embed Python expressions inside string literals for formatting. F-string is written as `f"string"`

OUTPUT

```
a = 'Kevin'
print(f"His name is {a}.")
```

His name is Kevin.

```
a, b = 5, 10
print(f"His age is {2 * (a + b)}.")
```

His age is 30.

The value of pi is: 3.14286

```
num = 3.1428571428
print(f"The value of pi is: {num : .5f}")
```

69

FORMATTING IN STRINGS

Formatting with center() string method.

The center() method is a built-in method in Python's str class that returns a new string that is centered within a string of a specified width. The string will be padded with spaces on the left and right sides

```
s = "Python Programming"
width = 30
centered_string = s.center(width)
print(centered_string)
```

OUTPUT

Python Programming

NEGATIVE INDICES

In a string, the last character can be accessed using the index -1, the second to last element using the index -2, and so on.

Characters	o	r	a	n	g	e		t	r	e	e
Index	0	1	2	3	4	5	6	7	8	9	10
-ve Index	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

```
s = "orange tree"
print(s[5])
Print(s[-6])
```

OUTPUT

```
e
e
```

STRING COMPARISON

Used to compare strings – to check whether 2 strings are the same or different

String comparison using relational operators: ==, !=

```
s1 = "rose"
s2 = 'rose'
s3 = 'roses'
print(s1 == s2)
print(s1 == s3)
if s2==s3:
    print(f"Strings {s2} and {s3} are the same")
else:
    print(f"Strings {s2} and {s3} are different")
```

OUTPUT

```
True
False
Strings rose and roses are different
```

STRING COMPARISON

Used to compare strings – to check whether 2 strings are the same or different

String comparison using relational operators: ==, !=

```
s1 = "rose"
s2 = 'rose'
s3 = 'roses'
print(s1 != s2)
print(s1 != s3)
if s1!=s2:
```

```
    print(f"Strings {s1} and {s2} are different")
```

```
else:
```

```
    print(f"Strings {s1} and {s2} are the same")
```

OUTPUT

False

True

Strings rose and roses are the same

73

STRING COMPARISON

Used to compare strings – to check whether 2 strings are the same or different

String comparison using identity operators: is, is not

```
s1 = 'greek'
s2 = 'greek'
s3 = 'latin'
print(id(s1))
print(id(s2))
print(id(s3))
print(s1 is s2)
print(s2 is s3)
```

OUTPUT

140105907644848

140105907644848

140105907647216

True

False

```
s2 = 'latin'
print(id(s2))
print(s1 is s2)
print(s2 is s3)
print(s1 is not s3)
print(s2 is not s3)
```

OUTPUT

140105907647216

False

True

True

False

STRING SLICING

- Slicing is a process of obtaining a sub-string from the given string by slicing it respectively from start to end.
- Python string is immutable, slicing creates a new substring from the source string and original string remains unchanged.
- Python slicing can be done using the array slicing `[ : : ]` method

75

STRING SLICING

SYNTAX:

```
str_object[start_pos : end_pos : step]
```

- `start_pos`: Starting index where the slicing of object starts.
- `stop_pos`: Ending index where the slicing of object stops.
- `step`: It is an optional argument that determines the increment between each index for slicing.
- **Return Type**: Returns a sliced object containing elements in the given range only.

76

STRING SLICING

Characters	o	r	a	n	g	e		f	r	u	i	t
Index	0	1	2	3	4	5	6	7	8	9	10	11
-ve Index	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

```

>>> s = "orange fruit"
>>> print(s[:])
orange fruit
>>> print(s[: :])
orange fruit

>>> i = 5
>>> print(s[: i] + s[i : ])
orange fruit
>>> print(s[: 5])
orang

```

77

STRING SLICING

Characters	o	r	a	n	g	e		f	r	u	i	t
Index	0	1	2	3	4	5	6	7	8	9	10	11
-ve Index	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

```

>>> print(s[3 : ])
nge fruit
>>> print(s[2 : 8])
ange f
>>> print(s[2 : 11 : 2])
ag ri

>>> print(s[30 : ])
>>> print(s[4 : 50])
ge fruit

```

78

STRING SLICING

Characters	o	r	a	n	g	e		f	r	u	i	t
Index	0	1	2	3	4	5	6	7	8	9	10	11
-ve Index	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

```
>>>print(s[ : :-1])          >>>print(s[-3 : -10])
tiurf egnaro                  urf egn
>>>print(s[8 : 1 : -1])      >>>print(s[-4 : -2])
rf eгна                      ge fru
>>>print(s[8 : 1 : -2])
r ga
```

79

STRING SPLITTING

- String splitting is a process of splitting a string into a list of strings after breaking the given string by the specified separator.
- It is done using split() method. split() method breaks down a string into a list of substrings using a chosen separator. Default separator is any whitespace.

SYNTAX:

`string.split(separator, maxsplit)`

- Separator → Optional. Specifies the separator to use when splitting the string. By default any whitespace is a separator.
- Maxsplit → Optional. Specifies how many splits to do. Default value is -1, which is "all occurrences". If maxsplit is specified, the list will have a maximum of maxsplit+1 items.

80

STRING SPLITTING

Examples:

```
text = "Apple is good for health"
print(text.split())
# default separator is whitespace
```

OUTPUT

```
['Apple', 'is', 'good', 'for', 'health']
```

```
text1 = 'one, two, three'
print(text1.split(','))
text2 = 'apple#banana#cherry#orange'
print(text2.split('#'))
```

OUTPUT

```
['one', ' two', ' three']
```

```
['apple', 'banana', 'cherry', 'orange']
```

STRING SPLITTING

Examples:

```
text = 'one, two, three, four, five, six'
print(text.split(',', 0))
print(text.split(',', 4))
print(text.split(',', 1))
```

OUTPUT

```
['one, two, three, four, five, six']
```

```
['one', ' two', ' three', ' four', ' five, six']
```

```
[one, 'two, three, four, five, six']
```

STRING SPLITTING

Examples: `split()` method is used to create a list of elements from a text

```
text = 'apple#banana#cherry#orange'
list1 = text.split('#')
print(list1)
```

OUTPUT

```
['apple', 'banana', 'cherry', 'orange']
```

Examples: `join()` method can be used to construct a text from a list elements

```
new_text = ', '.join(list1)
print(new_text)
apple, banana, cherry, orange
```

OUTPUT

```
apple, banana, cherry, orange
```

STRING SPLITTING

Examples: Python program to verify if the email id is a valid gmail id or not

```
text = input("Enter your email id: ")
list1 = text.split('@')
if list1[1] == 'gmail.com':
    print("Email id verified..")
else:
    print("It is not a valid gmail id ")
```

OUTPUT 1:

```
Enter your email id:
priyanka123@gmail.com
Email id verified..
```

OUTPUT 2:

```
Enter your email id:
priyanka123@gmail.com
It is not a valid gmail id
```

STRING SPLITTING

Examples: Python program to change a text

```
text = input("Enter a string: ")
list1 = list(text)
pos = int(input(("Enter the index of the character to be changed: ")))
ch = input("Enter the new character: ")
list1[pos] = ch
new_text = "".join(list1)
print(new_text)
```

OUTPUT :

```
Enter a string: sample
Enter the index of the character to be changed: 1
Enter the new character: i
sample
```

STRING STRIPPING

- Stripping is a process of removing extra whitespaces and specified characters from the start and from the end of the strip irrespective of how the parameter is passed.
- It is done using strip() method in Python.
- This method returns a copy of the string with both leading and trailing characters removed.

SYNTAX:

```
string.strip(characters)
```

STRING STRIPPING

Examples:

```
string = "    Hello, world!    "
new_string = string.strip()
print(new_string)
```

OUTPUT

Hello, world!

```
string1 = '  xxxx oooo hello xxox ee  '
print(string1.strip(' xoe'))
```

OUTPUT

hell

all <whitespace>,x,o,e characters in the left # and right of string are removed

87

STRING FUNCTIONS

Python has a set of built-in methods that can be used on strings

S.No.	Method	Description
1	capitalize()	Converts the first character to upper case
2	casefold()	Converts string into lower case
3	center(width)	Returns a centered string
4	count()	Returns the number of times a specified value occurs in a string
5	endswith()	Returns true if the string ends with the specified value
6	find()	Searches the string for a specified value and returns the position of where it was found
7	format()	Formats specified values in a string
8	index()	Searches the string for a specified value and returns the position of where it was found
9	isalnum()	Returns True if all characters in the string are alphanumeric
10	isalpha()	Returns True if all characters in the string are in the alphabet

8

STRING FUNCTIONS

Python has a set of built-in methods that can be used on strings

S.No.	Method	Description
11	isascii()	Returns True if all characters in the string are ascii characters
12	isdecimal()	Returns True if all characters in the string are decimals
13	isdigit()	Returns True if all characters in the string are digits
14	isidentifier()	Returns True if the string is an identifier
15	islower()	Returns True if all characters in the string are lower case
16	isnumeric()	Returns True if all characters in the string are numeric
17	isspace()	Returns True if all characters in the string are whitespaces
18	istitle()	Returns True if the string follows the rules of a title
19	isupper()	Returns True if all characters in the string are upper case
20	join()	Converts the elements of an iterable into a string
21	ljust()	Returns a left justified version of the string
22	lower()	Converts a string into lower case

STRING FUNCTIONS

Python has a set of built-in methods that can be used on strings

S.No.	Method	Description
23	replace()	Returns a string where a specified value is replaced with a specified value
24	rfind()	Searches the string for a specified value and returns the last position of where it was found
25	rindex()	Searches the string for a specified value and returns the last position of where it was found
26	rjust()	Returns a right justified version of the string
27	split()	Splits the string at the specified separator, and returns a list
28	startswith()	Returns true if the string starts with the specified value
29	strip()	Returns a trimmed version of the string
30	swapcase()	Swaps cases, lower case becomes upper case and vice versa

STRING FUNCTIONS

Python has a set of built-in methods that can be used on strings

S.No.	Method	Description
30	swapcase()	Swaps cases, lower case becomes upper case and vice versa
31	title()	Converts the first character of each word to upper case
32	upper()	Converts a string into upper case
33	zfill()	Fills the string with a specified number of 0 values at the beginning

91

STRING FUNCTIONS

Sample python program to demonstrate the string functions

```
text1 = 'python program'
text2 = 'INDIA'
text3 = 'praveena@gmail.com'
text4 = 'HDFC162'
print(text1.capitalize())
print(text2.casefold())
print(text1.center(20))
print('o is present { } times'.format(text1.count('o')))
print(text2.endswith('A'))
print(f'In {text1} "g" is present at position ',
      text1.find('g'))
print(f'In {text1} "a" is present at position ',
      text1.index('a'))
print(text4.isalnum())
print(text4.isalpha())
print(text1.replace('g', 'd'))
print(text2.lower())
```

OUTPUT:

```
Python program
india
  python program
o is present 2 times
True
In python program "g" is present at
position 10
In python program "a" is present at
position 12
True
False
python prodrum
india
```

92